

数据结构

第四章 串

串类型的定义
串的实现和表示
串的模式匹配算法

第一节 串类型的定义

在非数值处理、事务处理等问题常涉及到一系列的字符操作。计算机的硬件结构主要是反映数值计算的要求，因此，字符串的处理比具体数值处理复杂。本章讨论串的存储结构及几种基本的处理。

串的基本概念

- 串(字符串)：是零个或多个字符组成的有限序列。记作： $S = \text{"}a_1a_2a_3\ldots\text{"}$ ，其中 S 是串名， $a_i (1 \leq i \leq n)$ 是单个，可以是字母、数字或其它字符。
- 串值：双引号括起来的字符序列是串值。
- 串长：串中所包含的字符个数称为该串的长度。
- 空串(空的字符串)：长度为零的串称为空串，它不包含任何字符。
- 空格串(空白串)：构成串的所有字符都是空格的串称为空白串。

串的基本概念

注意：空串和空白串的不同，例如 “ ” 和 “” 分别表示长度为1的空白串和长度为0的空串。

子串 (substring)：串中任意个连续字符组成的子序列称为该串的子串，包含子串的串相应地称为主串。

子串的序号：将子串在主串中首次出现时的该子串的首字符对应在主串中的序号，称为子串在主串中的序号（或位置）。

例如，设有串**A**和**B**分别是：

A = “这是字符串”， **B** = “是”

则**B**是**A**的子串，**A**为主串。其中首次出现所对应的主串位置是3。因此，称**B**在**A**中的序号为3。

串的例子

- 特别地，空串是任意串的子串，任意串是其自身的子串。
- 称两个串是相等的，当且仅当这两个串的值相等。也就是说，只有当两个串的长度相等，并且各个对应位置的字符都相等时才相等。

a = 'WU',

b = 'HAN'

c = 'WUHAN'

d = 'WU HAN'

- 值得一提的是，串值必须用一对单引号括起来，但单引号本身不属于串，它的作用只是为了避免与变量名或数的常量混淆而已。

串的抽象数据类型定义

ADT String{

数据对象: $D = \{ a_i | a_i \in \text{CharacterSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系: $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作:

StrAssign(t, chars)

初始条件: **chars**是一个字符串常量。

操作结果: 生成一个值为**chars**的串**t**。

StrConcat(s, t)

初始条件: 串**s, t** 已存在。

操作结果: 将串**t**联结到串**s**后形成新串存放到**s**中。

StrLength(t)

初始条件: 字符串**t**已存在。

操作结果: 返回串**t**中的元素个数, 称为串长。

SubString (s, pos, len, sub)

初始条件: 串**s**, 已存在, $1 \leq \text{pos} \leq \text{StrLength}(s)$ 且 $0 \leq \text{len} \leq \text{StrLength}(s) - \text{pos} + 1$ 。

操作结果: 用**sub**返回串**s**的第**pos**个字符起长度为**len**的子串。

.....

} ADT String

串的操作

基本操作:

StrAssign (&T, chars)

初始条件:chars 是字符串常量。

操作结果:生成一个其值等于 chars 的串 T。

StrCopy (&T, S)

初始条件:串 S 存在。

操作结果:由串 S 复制得串 T。

StrEmpty (S)

初始条件:串 S 存在。

操作结果:若 S 为空串,则返回 TRUE,否则返回 FALSE。

StrCompare (S, T)

初始条件:串 S 和 T 存在。

操作结果:若 $S > T$,则返回值 > 0 ;

若 $S = T$,则返回值 $= 0$;

若 $S < T$,则返回值 < 0 。

StrLength (S)

初始条件:串 S 存在。

操作结果:返回 S 的元素个数,称为串的长度。

ClearString (&S)

初始条件:串 S 存在。

操作结果:将 S 清为空串。

Concat (&T, S1, S2)

初始条件:串 S1 和 S2 存在。

操作结果:用 T 返回由 S1 和 S2 联接而成的新串。

SubString (&Sub, S, pos, len)

初始条件:串 S 存在, $1 \leq pos \leq \text{StrLength}(S)$

且 $0 \leq len \leq \text{StrLength}(S) - pos + 1$ 。

操作结果:用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。

Index (S, T, pos)

初始条件:串 S 和 T 存在,T 是非空串,

$1 \leq pos \leq \text{StrLength}(S)$ 。

操作结果:若主串 S 中存在和串 T 值相同的子串,则返回它的主串 S 中第 pos 个字符之后第一次出现的位置;否则函数值为 0。

Replace (&S, T, V)

初始条件:串 S, T 和 V 存在,T 是非空串。

操作结果:用 V 替换主串 S 中出现的所有与 T 相等的不重叠的子串。

StrInsert (&S, pos, T)

初始条件:串 S 和 T 存在, $1 \leq pos \leq \text{StrLength}(S) + 1$ 。

操作结果:在串 S 的第 pos 个字符之前插入串 T。

StrDelete (&S, pos, len)

初始条件:串 S 存在, $1 \leq pos \leq \text{StrLength}(S) - len + 1$ 。

操作结果:从串 S 中删除第 pos 个字符起长度为 len 的子串。

DestroyString (&S)

初始条件:串 S 存在。

操作结果:串 S 被销毁。

串的基本操作

□ 最小操作子集：StrAssign、StrCompare、StrLength、Concat、SubString

```
int Index(String S, String T, int pos) {  
    if(pos > 0) {  
        n = StrLength(S); m = StrLength(T); i = pos;  
        while(i <= n - m + 1) {  
            SubString(sub, S, i, m);  
            if(StrCompare(sub, T) != 0) ++i;  
            else return i;  
        } //while  
    } // if  
    return 0;  
} //Index
```


串的存储表示和实现

串是一种特殊的线性表，其存储表示和线性表类似，但又不完全相同。串的存储方式取决于将要对串所进行的操作。串在计算机中有3种表示方式：

- ◆ **定长顺序存储表示**：将串定义成字符数组，利用串名可以直接访问串值。用这种表示方式，串的存储空间在编译时确定，其大小不能改变。
- ◆ **堆分配存储方式**：仍然用一组地址连续的存储单元来依次存储串中的字符序列，但串的存储空间是在程序运行时根据串的实际长度动态分配的。
- ◆ **块链存储方式**：是一种链式存储结构表示。

串的定长顺序存储表示

这种存储结构又称为串的顺序存储结构。是用一组连续的存储单元来存放串中的字符序列。所谓定长顺序存储结构，是直接使用定长的字符数组来定义，数组的上界预先确定。

定长顺序存储结构定义为：

```
#define MAX_STRLEN 256  
typedef struct  
{ char str[MAX_STRLEN] ;  
    int length;  
} StringType ;
```

串的联结操作

Status StrConcat (StringType s, StringType t)

```
/* 将串t联结到串s之后, 结果仍然保存在s中 */
{ int i, j ;
  if ((s.length+t.length)>MAX_STRLEN)
    Return ERROR ; /* 联结后长度超出范围 */
  for (i=0 ; i<t.length ; i++)
    s.str[s.length+i]=t.str[i] ; /* 串t联结到串s
    之后 */
  s.length=s.length+t.length ; /* 修改联结后的串
  长度 */
  return OK ;
}
```

求子串操作

```
Status SubString (StringType s, int pos, int
len, StringType *sub)
{ int k, j ;
  if
  (pos<1||pos>s.length||len<0||len>(s.len
gth-pos+1))
    return ERROR ; /* 参数非法 */
  sub->length=len ; /* 求得子串长度 */
  for (j=0, k=pos ; j<len ; k++, j++)
    sub->str[j]=s.str[k] ; /* 逐个字符复制求得子
    串 */
  return OK ;
}
```

串的堆分配存储表示

实现方法：系统提供一个空间足够大且地址连续的存储空间(称为“堆”)供串使用。可使用C语言的动态存储分配函数**malloc()**和**free()**来管理。

特点是：仍然以一组地址连续的存储空间来存储字符串值，但其所需的存储空间是在程序执行过程中动态分配，故是动态的，变长的。

串的堆式存储结构的类型定义

```
typedef struct
```

```
{ char *ch; /* 若非空，按长度分配，否则为NULL */
```

```
    int length; /* 串的长度 */
```

```
} HString ;
```

串的联结操作

```
Status Hstring *StrConcat(HString *T, HString *s1, HString *s2)
/* 用T返回由s1和s2联结而成的串 */
{ int k, j, t_len ;
  if (T.ch) free(T); /* 释放旧空间 */
  t_len=s1->length+s2->length ;
  if ((T.ch =(char *)malloc(sizeof((char)*t_len))==NULL)
      { printf("系统空间不够, 申请空间失败! \n");
        return ERROR ; }
  for (j=0 ; j<s->length; j++)
    T->ch[j]=s1->ch[j] ; /* 将串s复制到串T中 */

  for (k=s1->length, j=0 ; j<s2->length; k++, j++)

    T->ch[k]=s2->ch[j] ; /* 将串s2复制到串T中 */

  free(s1->ch) ;

  free(s2->ch) ;

  return OK ;

}
```

串的链式存储表示

串的链式存储结构和线性表的串的链式存储结构类似，采用单链表来存储串，结点的构成是：

- ◆ **data域**：存放字符，**data域**可存放的字符个数称为结点的大小；
- ◆ **next域**：存放指向下一结点的指针。

若每个结点仅存放一个字符，则结点的指针域就非常多，造成系统空间浪费，为节省存储空间，考虑串结构的特殊性，使每个结点存放若干个字符，这种结构称为块链结构。如图4-1是块大小为3的串的块链式存储结构示意图。

串的链式存储表示

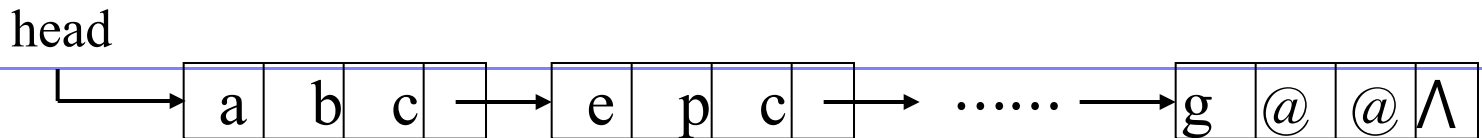


图4-1 串的块链式存储结构示意图

串的块链式存储的类型定义包括：

(1) 块结点的类型定义

```
#define BLOCK_SIZE 4
typedef struct Blstrtype
{ char data[BLOCK_SIZE];
  struct Blstrtype *next;
}BNODE ;
```

(2) 块链串的类型定义

```
typedef struct
{ BNODE head; /* 头指针 */
  int Strlen; /* 当前长度 */
} Blstring ;
```

在这种存储结构下，结点的分配总是完整的结点为单位，因此，为使一个串能存放在整数个结点中，在串的末尾填上不属于串值的特殊字符，以表示串的终结。

当一个块(结点)内存放多个字符时，往往会使操作过程变得较为复杂，如在串中插入或删除字符操作时通常需要在块间移动字符。

$$\text{存储密度} = \frac{\text{串值所占的存储位}}{\text{实际分配的存储位}}$$

串的模式匹配算法

模式匹配：子串在主串中的定位称为模式匹配或串匹配(字符串匹配)。模式匹配成功是指在主串S中能够找到模式串T，否则，称模式串T在主串S中不存在。

模式匹配的应用在非常广泛。例如，在文本编辑程序中，我们经常要查找某一特定单词在文本中出现的位置。显然，解此问题的有效算法能极大地提高文本编辑程序的响应性能。

模式匹配是一个较为复杂的串操作过程。迄今为止，人们对串的模式匹配提出了许多思想和效率各不相同的计算机算法。介绍两种主要的模式匹配算法。

Brute-Force模式匹配算法

设S为目标串，T为模式串，且不妨设：

$S = "s_0s_1s_2 \dots s_{n-1}"$ ， $T = "t_0t_1t_2 \dots t_{m-1}"$

串的匹配实际上是对合法的位置 $0 \leq i \leq n-m$ 依次将目标串中的子串 $s[i \dots i+m-1]$ 和模式串 $t[0 \dots m-1]$ 进行比较：

◆ 若 $s[i \dots i+m-1] = t[0 \dots m-1]$ ：则称从位置 i 开始的匹配成功，亦称模式 t 在目标 s 中出现；

◆ 若 $s[i \dots i+m-1] \neq t[0 \dots m-1]$ ：从 i 开始的匹配失败。

位置 i 称为位移，当 $s[i \dots i+m-1] = t[0 \dots m-1]$ 时， i 称为有效位移；当 $s[i \dots i+m-1] \neq t[0 \dots m-1]$ 时， i 称为无效位移。

串的模式匹配算法

这样，串匹配问题可简化为找出某给定模式T在给定目标串S中首次出现的有效位移。

```
int IndexString(StringType s , StringType t , int pos )
```

```
/* 采用顺序存储方式存储主串s和模式t, */
```

```
/* 若模式t在主串s中从第pos位置开始有匹配的子串, */
```

```
/* 返回位置，否则返回-1 */
```

```
{ char *p , *q ;
```

```
int k , j ;
```

```
k=pos-1 ; j=0 ; p=s.str+pos-1 ; q=t.str ;
```

```
/* 初始匹配位置设置 */
```

```
/* 顺序存放时第pos位置的下标值为pos-1 */
```

串的模式匹配算法

```
while (k<s.length && j<t.length)
{
    if (*p==*q) { p++ ; q++ ; k++ ; j++ ; }
    else { k=k-j+1 ; j=0 ; q=t.str ; p=s.str+k ; }
    /* 重新设置匹配位置 */
}
if (j==t.length)
    return(k-t.length) ; /* 匹配, 返回位置 */
else return(-1) ; /* 不匹配, 返回-1 */
}
```

串的模式匹配算法

该算法简单，易于理解。在一些场合的应用里，如文字处理中的文本编辑，其效率较高。

该算法的时间复杂度为 $O(n*m)$ ，其中 n 、 m 分别是主串和模式串的长度。通常情况下，实际运行过程中，该算法的执行时间近似于 $O(n+m)$ 。

理解该算法的关键点

当第一次 $s_k \neq t_j$ 时：主串要退回到 $k-j+1$ 的位置，而模式串也要退回到第一个字符（即 $j=0$ 的位置）。

比较出现 $s_k \neq t_j$ 时：则应该有 $s_{k-1} = t_{j-1}$ ， $\dots$ ， $s_{k-j+1} = t_1$ ， $s_{k-j} = t_0$ 。

模式匹配的一种改进算法

该改进算法是由D.E.Knuth，J.H.Morris和 V.R.Pratt提出来的，简称为KMP算法。其改进在于：

每当一趟匹配过程出现字符不相等时，主串指示器不用回溯，而是利用已经得到的“部分匹配”结果，将模式串的指示器向右“滑动”尽可能远的一段距离后，继续进行匹配。

例：设有串s="abacabab"，t="abab"。则第一次匹配过程如下图所示。

s="a b a c a b a b"	i=4	
≠		匹配失败
t="a b a b"	j=4	

图 模式匹配示例

模式匹配的一种改进算法

在 $i=4$ 和 $j=4$ 时，匹配失败。但重新开始第二次匹配时，不必从 $i=2$ ， $j=1$ 开始。第二次匹配可以直接从 $i=4$ 、 $j=2$ 开始。

总之，在主串 s 与模式串 t 的匹配过程中，一旦出现 $s_i \neq t_j$ ，主串 s 的指针不必回溯，而是直接与模式串的 t_k ($1 \leq k < j$ 进行比较，而 k 的取值与主串 s 无关，只与模式串 t 本身的构成有关，即从模式串 t 可求得 k 值。)

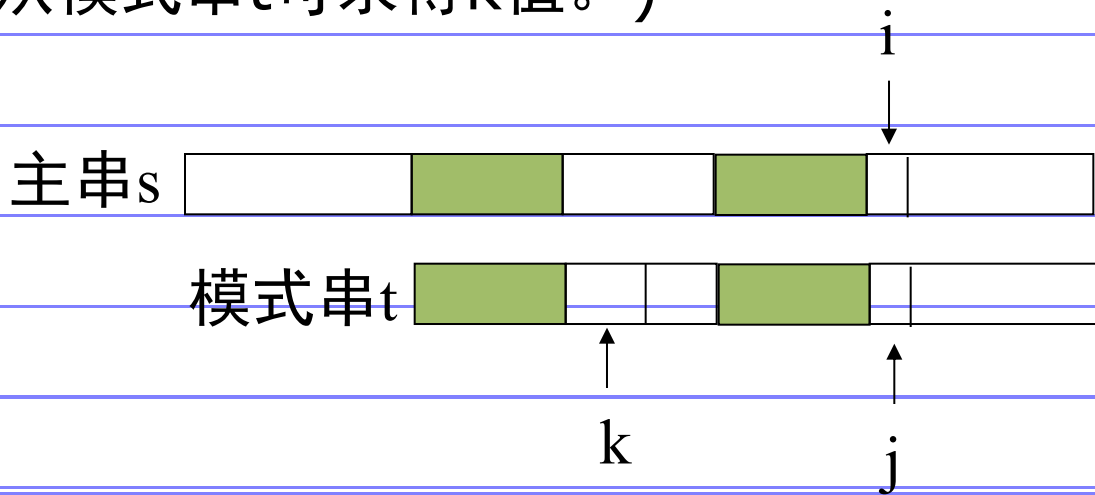


图4-3 KMP算法示例

KMP算法流程

BBC ABCDAB ABCDABCDABDE
ABCDABD

<https://blog.csdn.net/daaikuaichuan>

BBC ABCDAB ABCDABCDABDE
ABCDABD

<https://blog.csdn.net/daaikuaichuan>

BBC ABCDAB ABCDABCDABDE
ABCDABD

<https://blog.csdn.net/daaikuaichuan>

KMP算法流程

i	0	1	2	3	4	5	6	7
模式串	A	B	C	D	A	B	D	'\0'
next[i]	-1	0	0	0	0	1	2	0

BBC ABCDAB ABCDABCDABDE

ABCDABD

KMP算法流程

字符串: "china"

真前缀: c, ch, chi, chin

真后缀: hina, ina, na, a

<https://blog.csdn.net/daaikuaichuan>

next数组的求解基于“真前缀”和“真后缀”

next[i] 等于P[0]~P[i-1]最长的相同真前后缀的长度

如果给定的模式串是：“ABCDABD”，从左至右遍历整个模式串，其各个子串的前缀后缀分别如下表格所示：

模式串的各个子串	前缀	后缀	最大公共元素长度
A	空	空	0
AB	A	B	0
ABC	A,AB	C,BC	0
ABCD	A,AB,ABC	D,CD,BCD	0
ABCD A	A,AB,ABC,ABCD	A,DA,CDA,BCDA	1
ABCDAB	A,AB,ABC,ABCD,ABCD A	B,AB,DAB,CDAB,BCDAB	2
ABCDABD	A,AB,ABC,ABCD,ABCD A ABCDAB	D,BD,ABD,DABD,CDABD BCDABD	0

<https://blog.csdn.net/daaikuaichuan>

定义next[j]函数为

$$\text{next}[j] = \begin{cases} 0 & \text{当 } j=1 \text{ 时} \\ \text{Max}\{k | 1 < k < j \wedge t_1 t_2 \dots t_{k-1} = t_{j-(k-1)} t_{j-k} \dots t_{j-1}\} & \text{该集合不空时} \\ 1 & \text{其它情况} \end{cases}$$

在求得了 $\text{next}[j]$ 值之后，KMP算法的思想是：

设目标串(主串)为 s ，模式串为 t ，并设 i 指针和 j 指针分别指示目标串和模式串中正待比较的字符，设 i 和 j 的初值均为1。若有 $s_i = t_j$ ，则 i 和 j 分别加1。否则， i 不变， j 退回到 $j = \text{next}[j]$ 的位置，再比较 s_i 和 t_j ，若相等，则 i 和 j 分别加1。否则， i 不变， j 再次退回到 $j = \text{next}[j]$ 的位置，依此类推。直到下列两种可能：

- (1) j 退回到某个下一个 $[j]$ 值时字符比较相等，则指针各自加1继续进行匹配。
- (2) 退回到 $j = 0$ ，将 i 和 j 分别加1，即从主串的下一个字符 s_{i+1} 模式串的 t_1 重新开始匹配。

模式匹配的一种改进算法

```
#define Max_Strlen 1024
int next[Max_Strlen];
int KMP_index (StringType s , StringType t)
    /* 用KMP算法进行模式匹配，匹配返回位置，否则返回0 */
    /*用静态存储方式保存字符串， s和t分别表示主串和模式串 */
{   int  k=1 , j=1 ;    /*初始匹配位置设置 */
    while ( k≤s.length && j ≤ t.length )
    {
        if (j==0 || s. str[k-1]==t.str[j-1]) { k++ ; j++ ; }
        else j=next[j-1] ;
    }
    if (j> t.length) return(k-t.length+1) ;
    else return(0) ;
}
```

模式匹配的一种改进算法

根据上述分析，求next函数值的算法如下：

```
void next(StringType t, int next[])
/* 求模式t的next串t函数值并保存在next数组中 */
{   int k=0, j=1; next[j-1]=0;
    while (j < t.length)
        {   if (k==0 || t.str[k-1]==t.str[j-1])
                {   k++; j++;
                    next[j-1]=k-1;
                }
            else
                k = next[k-1];
        }
}
```

习题四

(1) 解释下列每对术语的区别：空串和空白串；主串和子串；目标串和模式串。

(2) 若x和y是两个采用顺序结构存储的串，写一算法比较这两个字符串是否相等。

(3) 写一算法void StrRelace(char *T, char *P, char *S)，将T中第一次出现的与P相等的子串替换为S，串S和P的长度不一定相等，并分析时间复杂度。